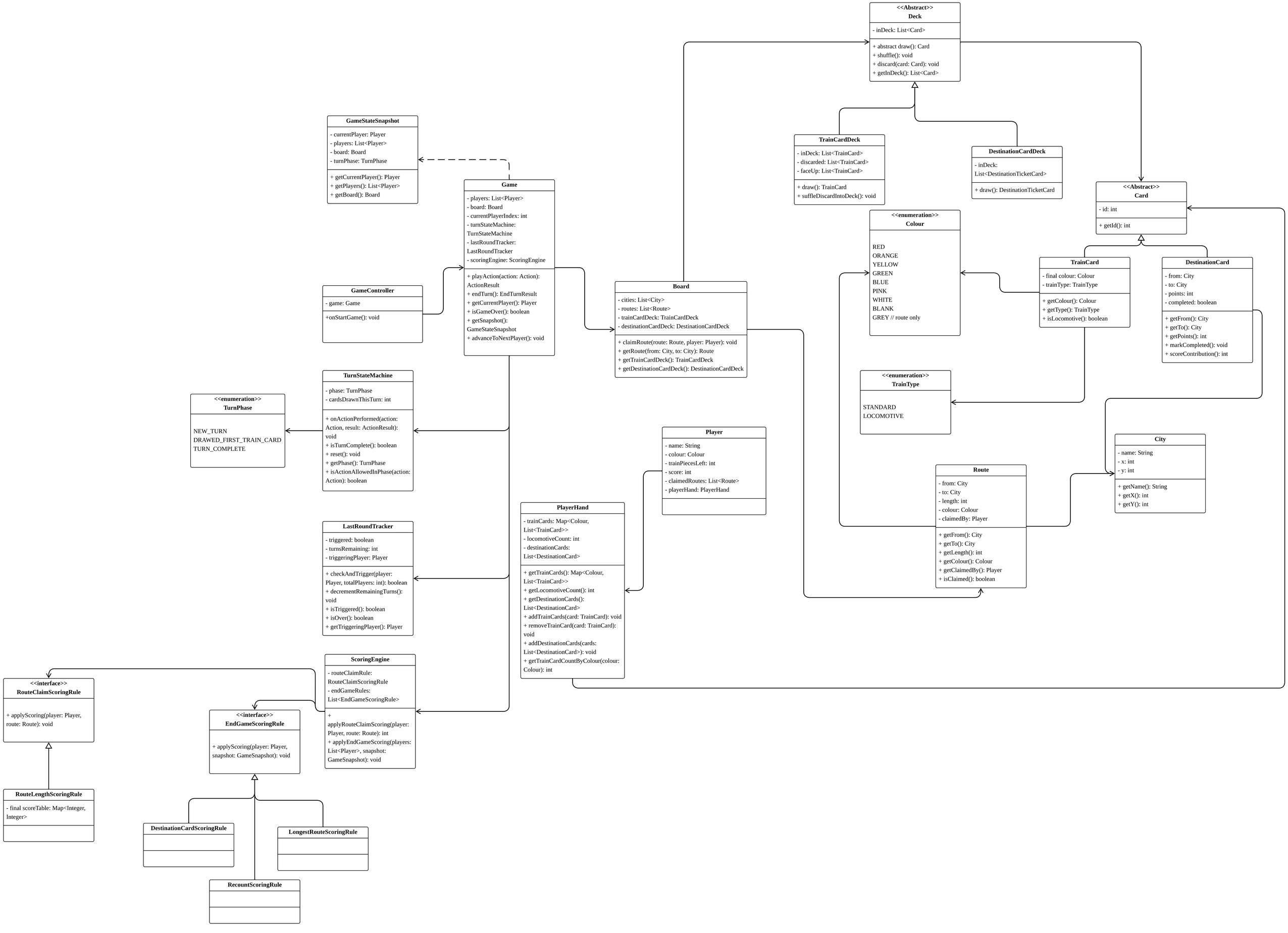

<<Abstract>>
Deck
- inDeck: List<Card>
+ abstract draw(): Card
+ shuffle(): void
+ discard(card: Card): void
+ getInDeck(): List<Card>
TrainCardDeck
- inDeck: List<TrainCard>
- discarded: List<TrainCard>
- faceUp: List<TrainCard>
+ draw(): TrainCard
+ suffleDiscardIntoDeck(): void
DestinationCardDeck
- inDeck: List<DestinationTicketCard>
+ draw(): DestinationTicketCard
GameStateSnapshot
- currentPlayer: Player
- players: List<Player>
- board: Board
- turnPhase: TurnPhase
+ getCurrentPlayer(): Player
+ getPlayers(): List<Player>
+ getBoard(): Board
Game
- players: List<Player>
- board: Board
- currentPlayerIndex: int
- turnStateMachine: TurnStateMachine
- lastRoundTracker: LastRoundTracker
- scoringEngine: ScoringEngine
+ playAction(action: Action): ActionResult
+ endTurn(): EndTurnResult
+ getCurrentPlayer(): Player
+ isGameOver(): boolean
+ getSnapshot(): GameStateSnapshot
+ advanceToNextPlayer(): void
GameController
- game: Game
+onStartGame(): void
Board
- cities: List<City>
- routes: List<Route>
- trainCardDeck: TrainCardDeck
- destinationCardDeck: DestinationCardDeck
+ claimRoute(route: Route, player: Player): void
+ getRoute(from: City, to: City): Route
+ getTrainCardDeck(): TrainCardDeck
+ getDestinationCardDeck(): DestinationCardDeck
<<enumeration>>
Colour
RED
ORANGE
YELLOW
GREEN
BLUE
PINK
WHITE
BLANK
GREY // route only
<<Abstract>>
Card
- id: int
+ getId(): int
TrainCard
- final colour: Colour
- trainType: TrainType
+ getColour(): Colour
+ getType(): TrainType
+ isLocomotive(): boolean
DestinationCard
- from: City
- to: City
- points: int
- completed: boolean
+ getFrom(): City
+ getTo(): City
+ getPoints(): int
+ markCompleted(): void
+ scoreContribution(): int
<<enumeration>>
TrainType
STANDARD
LOCOMOTIVE
TurnStateMachine
- phase: TurnPhase
- cardsDrawnThisTurn: int
+ onActionPerformed(action: Action, result: ActionResult): void
+ isTurnComplete(): boolean
+ reset(): void
+ getPhase(): TurnPhase
+ isActionAllowedInPhase(action: Action): boolean
<<enumeration>>
TurnPhase
NEW_TURN
DRAWED_FIRST_TRAIN_CARD
TURN_COMPLETE
Player
- name: String
- colour: Colour
- trainPiecesLeft: int
- score: int
- claimedRoutes: List<Route>
- playerHand: PlayerHand
Route
- from: City
- to: City
- length: int
- colour: Colour
- claimedBy: Player
+ getFrom(): City
+ getTo(): City
+ getLength(): int
+ getColour(): Colour
+ getClaimedBy(): Player
+ isClaimed(): boolean
City
- name: String
- x: int
- y: int
+ getName(): String
+ getX(): int
+ getY(): int
PlayerHand
- trainCards: Map<Colour, List<TrainCard>>
- locomotiveCount: int
- destinationCards: List<DestinationCard>
+ getTrainCards(): Map<Colour, List<TrainCard>>
+ getLocomotiveCount(): int
+ getDestinationCards(): List<DestinationCard>
+ addTrainCards(card: TrainCard): void
+ removeTrainCard(card: TrainCard): void
+ addDestinationCards(cards: List<DestinationCard>): void
+ getTrainCardCountByColour(colour: Colour): int
LastRoundTracker
- triggered: boolean
- turnsRemaining: int
- triggeringPlayer: Player
+ checkAndTrigger(player: Player, totalPlayers: int): boolean
+ decrementRemainingTurns(): void
+ isTriggered(): boolean
+ isOver(): boolean
+ getTriggeringPlayer(): Player
ScoringEngine
- routeClaimRule: RouteClaimScoringRule
- endGameRules: List<EndGameScoringRule>
+ applyRouteClaimScoring(player: Player, route: Route): int
+ applyEndGameScoring(players: List<Player>, snapshot: GameSnapshot): void
<<interface>>
RouteClaimScoringRule
+ applyScoring(player: Player, route: Route): void
<<interface>>
EndGameScoringRule
+ applyScoring(player: Player, snapshot: GameSnapshot): void
RouteLengthScoringRule
- final scoreTable: Map<Integer, Integer>
DestinationCardScoringRule
LongestRouteScoringRule
RecountScoringRule